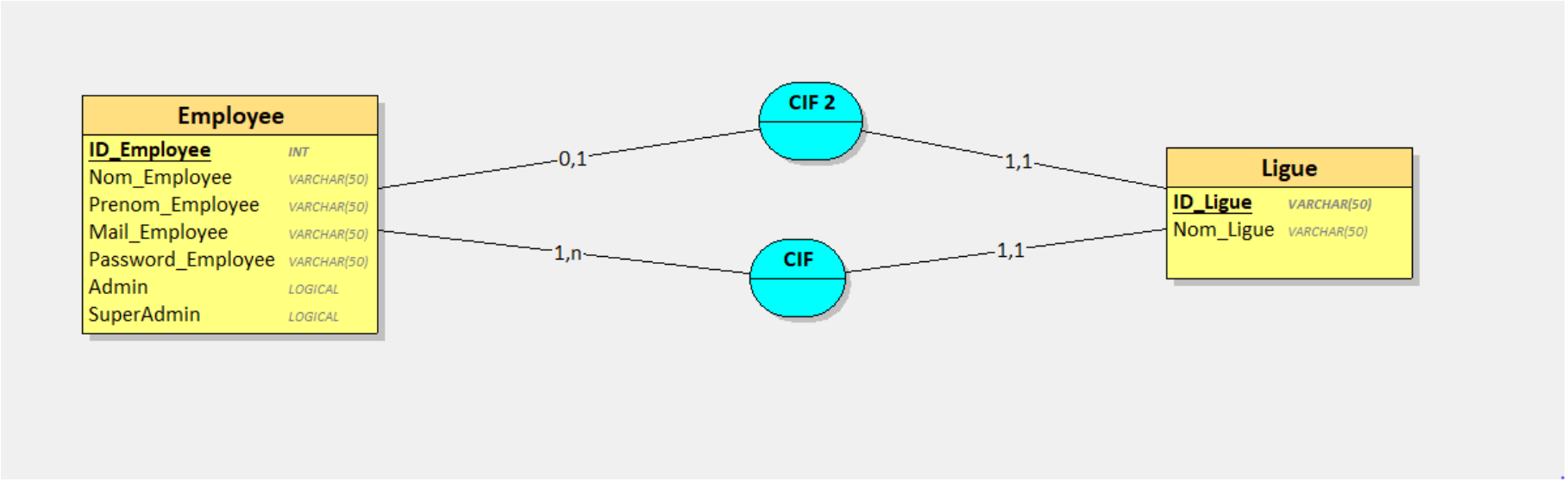
# Compte Rendu Itération

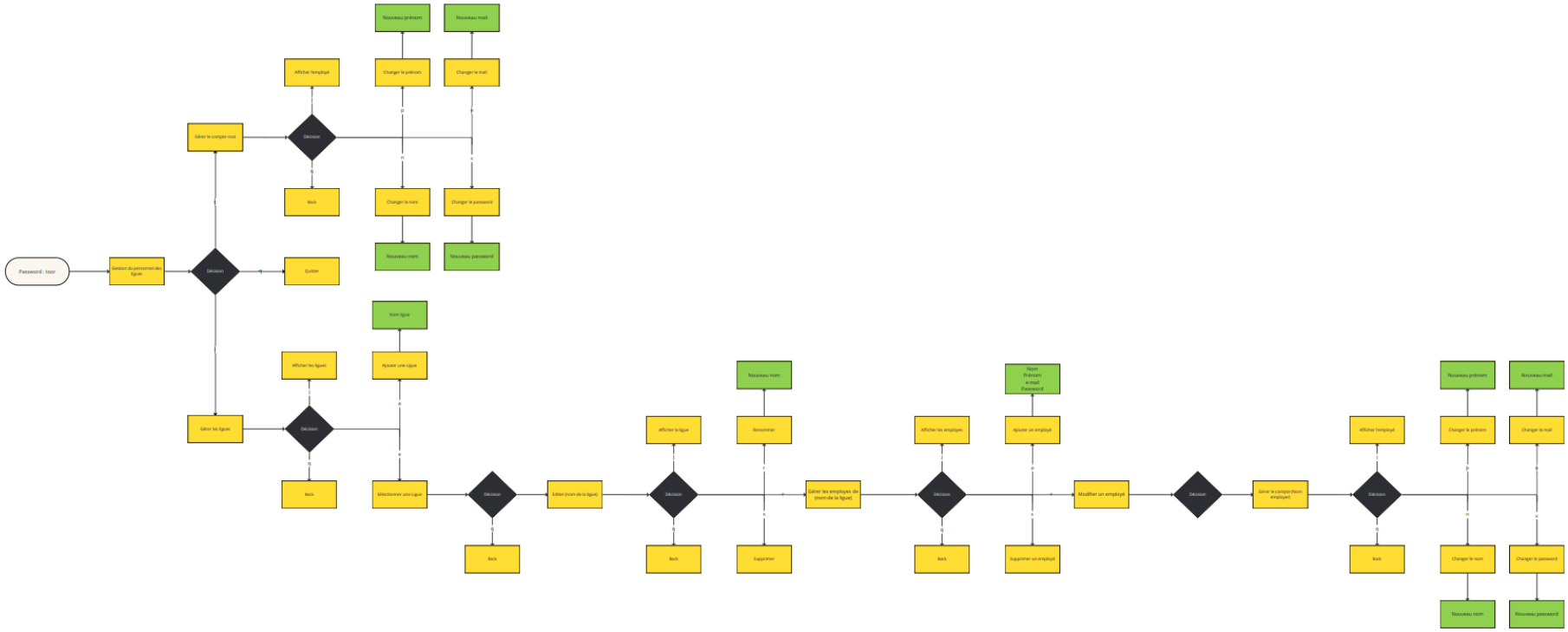
## Création du MCD :

Le MCD présente deux entités principales : Employee et Ligue. L'entité Employee contient des attributs tels que l'ID, le nom, le prénom, l'e-mail, le mot de passe, et des champs pour indiquer les droits d'admin et de super-admin. L'entité Ligue comprend l'ID et le nom de la ligue. Deux relations sont définies : l'une est une relation de **1,n** entre Employee et Ligue, indiquant qu'un employé peut être associé à plusieurs ligues ; l'autre est une relation **0,1** à **1,1**, suggérant qu'une ligue peut être optionnellement liée à un employé spécifique. Ce MCD nous a permis de visualiser les entités et leurs relations dans le système, ce qui nous a faciliter la conception de la maquette.



## Création de la maquette :

La maquette présente le système de gestion destiné aux ligues et au personnel associé. Il propose plusieurs fonctionnalités, telles que la gestion du compte root, la modification des informations des employés (nom, prénom, e-mail, mot de passe), et la gestion des ligues (ajout, suppression, et modification du nom des ligues). Les utilisateurs peuvent naviguer à travers des options pour afficher les ligues et les employés, ajouter ou supprimer des employés, ainsi qu'éditer les détails des comptes. Chaque action est décrite avec des choix précis permettant d'effectuer différentes opérations, structurées autour de décisions claires pour la gestion des données et des utilisateurs. (Voir GitHub pour une meilleure qualité.)



## Création du script des tables SQL :

Nous avons utilisé le MCD réalisé dans Itinération-1 pour réaliser ce script des tables SQL. Ce script sert à définir la structure de la base de données pour gérer le système de ligues et d'employés. Voici ses objectifs principaux :

**Stocker les ligues :** La table *Ligue* permet de garder les informations de chaque ligue, tels que son identifiant ou encore son nom, ce qui permet d'identifier et de nommer chaque ligue.

**Stocker les employés :** La table *Employe* garde les informations de chaque employé, comme leur prénom, nom email ou encore mot de passe. Leur statut admin y est aussi défini, et chaque employé est lié à une ligue grâce à la clé étrangère ID_ligue.

**Gestion des relations :** Une contrainte permet qu'un employé ne peut être associé à une seule et même ligue, ce qui garanti que chaque employé est toujours associé à une ligue existante.

Tout cela permet l'efficacité de la gestion des ligues, ddes employés et de leur relation tout en étant cohérent.

```sql
CREATE TABLE Ligue (
    ID_Ligue INT,
    nomLigue VARCHAR(100) NOT NULL,
    ADD CONSTRAINT PK_Ligue PRIMARY KEY (ID_Ligue)
)engine = innodb;

CREATE TABLE Employe (
    ID_Employe INT,
    prenomEmploye VARCHAR(50) NOT NULL,
    nomEmploye VARCHAR(50) NOT NULL,
    mail VARCHAR(100) UNIQUE NOT NULL,
    passwd VARCHAR(50) NOT NULL,
    datearv DATE NOT NULL,
    datedepart DATE,
    Admin BOOLEAN NOT NULL,
    ID_Ligue INT,
    ADD CONSTRAINT PK_Employe PRIMARY KEY (ID_Employe)
)engine = innodb;

ALTER TABLE Employe
ADD CONSTRAINT fk_Ligue_employe
FOREIGN KEY (ID_Ligue)
REFERENCES Ligue(ID_Ligue);
```

Lien du script : [Iteration2/CREATE TABLE.sql at main · WassimElArche/Iteration2]

## Modification du Code en Java :

Le code LigueConsole propose une interface en ligne qui permet d'interagir avec l'utilisateur. Celui-ci permet de gérer des ligues et leurs employés. Voici les principales fonctionnalités :

**Menu principal :** Afficher les ligues, ajouter une ligue et sélectionner une ligue.

```
Menu menuLigues()
{
        Menu menu = new Menu("Gérer les ligues", "l");
        menu.add(afficherLigues());
        menu.add(ajouterLigue());
        menu.add(selectionnerLigue());
        menu.addBack("q");
        return menu;
}
```

**Gestion des ligues :** Afficher les détails d'une ligue, renommer une ligue, changer l'administrateur et supprimer une ligue.

```
Menu menuLigues()
{
        Menu menu = new Menu("Gérer les ligues", "l");
        menu.add(afficherLigues());
        menu.add(ajouterLigue());
        menu.add(selectionnerLigue());
        menu.addBack("q");
        return menu;
}
```

**Gestion des employés :** Afficher les employés, ajouter un employé, modifier un employé et supprimer un employé.

```
private Menu gererEmployes(Ligue ligue)
{
        Menu menu = new Menu("Gérer les employés de " + ligue.getNom(), "e");
        menu.add(afficherEmployes(ligue));
        menu.add(ajouterEmploye(ligue));
        menu.add(selectEmploye(ligue));
        menu.addBack("q");
        return menu;
}
```

**Gestion des erreurs :** Affichage de messages en cas d'erreurs et contrôle les formats.

Cela permet une gestion simple et complète des ligues et des employés pour l'utilisateur.

Lien du code : personnel/Personnel/src/commandLine/LigueConsole.java at master · WassimElArche/personnel

## Création du code JDBC :

Le code présente une classe JDBC qui implémente l'interface Passerelle, permettant de gérer des opérations de base de données (BD) concernant des employés et des ligues via JDBC. Elle établit une connexion à la base de données en utilisant les informations contenues dans la classe Credentials. Voici ces principales fonctionnalités :

**JDBC :** comme dit précédemment, il permet d'établir une connexion à la base de données.

```
public JDBC()
{
        try
        {
                Class.forName(Credentials.getDriverClassName());
                connection = DriverManager.getConnection(Credentials.getUrl(), Credentials.getUser(), Credentials.getPassword());
        }
        catch (ClassNotFoundException e)
        {
                System.out.println("Pilote JDBC non installé.");
        }
        catch (SQLException e)
        {
                System.out.println(e);
        }
```

**GestionPersonnel :** Permet de récupérer les données des employés et des ligues depuis la base de données et les associer pour créer un objet à son nom (ligne 36 à 100).

**Insert :** Permet d'ajouter un employé ou une ligue à la base de données.

**Update :** Permet de modifier un employé ou une ligue à la base de données.

**Delete :** Permet de supprimer un employé ou une ligue à la base de données.

**SauvegarderGestionPersonnel :** Permet la fermeture de la connexion à la base de données.

```
@Override
public void sauvegarderGestionPersonnel(GestionPersonnel gestionPersonnel) throws SauvegardeImpossible
{
        close();
}

public void close() throws SauvegardeImpossible
{
        try
        {
                if (connection != null)
                        connection.close();
        }
        catch (SQLException e)
        {
                throw new SauvegardeImpossible(e);
        }
}
```

## Création de la maquette pour l’interface graphique :

La création de la maquette permet de visualiser l’interface de l’application pour ce projet. L’interface graphique est une représentation de la maquette créée ci-dessus, permettant la visualisation des fonctionnalités de tout les codes ainsi que de leurs outils.

Lien de toute l’interface graphique : https://github.com/WassimElArche/personnel/blob/master/Personnel/Maquette%20graphique.pdf